

CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

RocketTokenRETH

0xae78736Cd615f374D3085123A210448E74Fc6393

Ethereum

Compiler: v0.7.6+commit.7338295f

Date: October 7, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

Medium

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. RocketTokenRETH implements the exchange-rate-pegged rETH token for Rocket Pool, delegating balance/price data to an external oracle contract (RocketNetworkBalances) and access control for minting to a dynamically-registered RocketDepositPool contract. The core accounting logic (mul-before-div ordering, deposit delay before transfers) is sound and no classic reentrancy or arithmetic flaws were found. The main residual risks are architectural: (1) use of the legacy `.transfer()` pattern for ETH payouts in `burn()`, and (2) the fact that minting authority is gated entirely by a mutable address stored in the external RocketStorage registry, making the mint path's security dependent on the integrity of Rocket Pool's governance/guardian system rather than this contract itself.

Findings (4)

F-001

Medium

Minting authority and collateral routing depend entirely on a mutable external registry entry

/contracts/contract/token/RocketTokenRETH.sol:103–128

DESCRIPTION

`mint()` and `depositExcess()` are gated by `onlyLatestContract("rocketDepositPool", msg.sender)`, which resolves the authorized caller by looking up `contract.address` + "rocketDepositPool"` in the external `RocketStorageInterface``. There is no on-chain pinning, timelock, or multi-step verification in this contract itself, any address that RocketStorage's guardian/DAO registers under the name `rocketDepositPool`` immediately gains unrestricted ability to call `mint(_ethAmount, _to)``, which mints rETH 1:1 against the oracle-reported exchange rate with no cap check against actual deposited ETH.

IMPACT

If the guardian/DAO-controlled RocketStorage registry is compromised, mis-configured, or the upgrade process is socially-engineered, an attacker-controlled contract can be registered as `rocketDepositPool`` and mint unlimited rETH, which can then be redeemed via `burn()` against real collateral held by the protocol, resulting in a complete drain of backing ETH.

RECOMMENDATION

Document and enforce strict governance controls (timelocks, multi-sig thresholds, on-chain upgrade delay) around changes to the `contract.address`` mapping for `rocketDepositPool`` in RocketStorage. Consider adding a secondary sanity check in `mint()` (e.g., a per-block/per-epoch mint cap, or invariant check against `getTotalCollateral()/outstanding deposits`) so a single compromised registry entry cannot mint unbounded rETH in one transaction.

```
function mint(uint256 _ethAmount, address _to) override external onlyLatestContract("rocketDepositPool", msg.sender) {
    uint256 rethAmount = getRethValue(_ethAmount);
    require(rethAmount > 0, "Invalid token mint amount");
    _mint(_to, rethAmount);
    emit TokensMinted(_to, rethAmount, _ethAmount, block.timestamp);
}
```

ETH payout in burn() uses transfer() with a hardcoded 2300 gas stipend

/contracts/contract/token/RocketTokenRETH.sol:117–123

DESCRIPTION

`burn()` sends redeemed ETH to `msg.sender` using `msg.sender.transfer(ethAmount)`. Solidity's `transfer()` forwards a hardcoded 2300 gas stipend, which is insufficient for recipients whose receive/fallback logic needs more gas (e.g. multisig wallets, proxy wallets with storage writes, or EIP-1271-style smart-contract accounts). This gas cost may also change with future EVM opcode re-pricing, which has broken `.transfer()` compatibility in the past (e.g. EIP-2929).

IMPACT

Smart-contract wallet holders of rETH (e.g. Gnosis Safe without a gas-efficient fallback, or any contract with non-trivial receive logic) can be permanently unable to call `burn()` directly and redeem their rETH for ETH, resulting in a denial of service for that redemption path (users would need to transfer rETH to an EOA first).

RECOMMENDATION

Replace `msg.sender.transfer(ethAmount)` with a low-level call forwarding all available gas and checking the return value, guarded by checks-effects-interactions (already satisfied since `_burn` occurs beforehand):

```
```solidity
(bool success,) = msg.sender.call{value: ethAmount}("");
require(success, "ETH transfer failed");
```
```

```
_burn(msg.sender, _rethAmount);
withdrawDepositCollateral(ethAmount);
msg.sender.transfer(ethAmount);
emit TokensBurned(msg.sender, _rethAmount, ethAmount, block.timestamp);
```

Exchange rate depends on periodically-submitted oracle balances with no staleness guard in this contract

/contracts/contract/token/RocketTokenRETH.sol:61–74

DESCRIPTION

`getEthValue`/`getRethValue`/`getCollateralRate` all rely on `RocketNetworkBalancesInterface.getTotalETH-Balance()` / `getTotalRETHSupply()`, values that are only updated when trusted node operators submit balance reports (`submitBalances`/`executeUpdateBalances`), not continuously. This contract itself performs no staleness check (e.g. comparing `getBalancesBlock()` to `block.number`) before using these values for mint/burn pricing.

IMPACT

During periods of reporting delay or if balance submission consensus stalls, mint/burn pricing could reflect outdated network state. The existing `network.reth.deposit.delay` mechanism in `_beforeTokenTransfer` mitigates the most severe front-running pattern (deposit-then-immediately-transfer-around-an-update), but it does not address general staleness risk.

RECOMMENDATION

Consider surfacing `getBalancesBlock()`/staleness information to consumers, or add a sanity bound in `getEthValue`/`getRethValue` reverting if the last balance submission is older than an acceptable threshold, to avoid using very stale oracle data for pricing.

```
function getEthValue(uint256 _rethAmount) override public view returns (uint256) {
    RocketNetworkBalancesInterface rocketNetworkBalances = RocketNetworkBalancesInter-
    face(getContractAddress("rocketNetworkBalances"));
    uint256 totalEthBalance = rocketNetworkBalances.getTotalETHBalance();
    uint256 rethSupply = rocketNetworkBalances.getTotalRETHSupply();
    if (rethSupply == 0) { return _rethAmount; }
    return _rethAmount.mul(totalEthBalance).div(rethSupply);
}
```

F-004 · SWC-135

Informational

Redundant balance check duplicated with internal `_burn` logic

/contracts/contract/token/RocketTokenRETH.sol:110–111

DESCRIPTION

`_burn()` explicitly checks `require(balanceOf(msg.sender) >= _rethAmount, ...)` immediately before calling `_burn`, which internally performs the same check via `SafeMath.sub` with its own revert message ("ERC20: burn amount exceeds balance"). The explicit check is functionally redundant.

IMPACT

No security impact; purely a minor gas/code-clarity issue (an extra SLOAD + comparison on every burn call).

RECOMMENDATION

Remove the redundant `require(balanceOf(msg.sender) >= _rethAmount, ...)` check and rely on `_burn`'s built-in balance check, or keep only one with a clearer custom message if desired for UX.

```
require(_rethAmount > 0, "Invalid token burn amount");
require(balanceOf(msg.sender) >= _rethAmount, "Insufficient rETH balance");
uint256 ethAmount = getEthValue(_rethAmount);
```

Gas Optimizations

Remove redundant balance check before `_burn`

/contracts/contract/token/RocketTokenRETH.sol:111–undefined

Eliminate the explicit `balanceOf(msg.sender) >= _rethAmount` require since `_burn` already reverts on insufficient balance via `SafeMath` subtraction, saving one `SLOAD` + comparison per burn call.

Cache repeated `address(this).balance` reads

/contracts/contract/token/RocketTokenRETH.sol:150–161

In `depositExcessCollateral()`, `address(this).balance` is read multiple times (in `getCollateralRate()` indirectly, and again directly). Cache it into a local variable at the start of the function to avoid repeated `BALANCE` opcodes within the same transaction.

Mark helper getter functions external where not called internally

/contracts/contract/token/RocketTokenRETH.sol:80–83

`getExchangeRate()` is already external, but review other `public` view functions (e.g. `getTotalCollateral()`) that are not invoked internally elsewhere and mark them `external` to save minor calldata-copy gas on external calls.

Appendix - Metadata

| | |
|----------|--|
| Contract | RocketTokenRETH |
| Address | 0xae78736Cd615f374D3085123A210448E74Fc6393 |
| Chain | Ethereum |
| Compiler | v0.7.6+commit.7338295f |
| Audit ID | portfolio-rocketpool-reth |
| Date | 10/7/2026 |

Files analyzed

```
[dependency] @openzeppelin/contracts/math/SafeMath.sol
[dependency] @openzeppelin/contracts/token/ERC20/ERC20.sol
[dependency] @openzeppelin/contracts/token/ERC20/IERC20.sol
[dependency] @openzeppelin/contracts/utils/Context.sol
[user] /contracts/contract/RocketBase.sol
[user] /contracts/contract/token/RocketTokenRETH.sol
[user] /contracts/interface/RocketStorageInterface.sol
[user] /contracts/interface/dao/protocol/settings/RocketDAOProtocolSettingsNetwork-
Interface.sol
[user] /contracts/interface/deposit/RocketDepositPoolInterface.sol
[user] /contracts/interface/network/RocketNetworkBalancesInterface.sol
[user] /contracts/interface/token/RocketTokenRETHInterface.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

\$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

[Request Manual Audit '](https://cryptohawking.com/audit/manual?ref=portfolio-rocketpool-reth)

<https://cryptohawking.com/audit/manual?ref=portfolio-rocketpool-reth>

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

<https://cryptohawking.com/services>

WhatsApp (preferred): +92 322 4274236, <https://wa.me/923224274236>